

Default ASP.NET Core Web API Files and Folders

Back to: [ASP.NET Core Web API Tutorials](#)

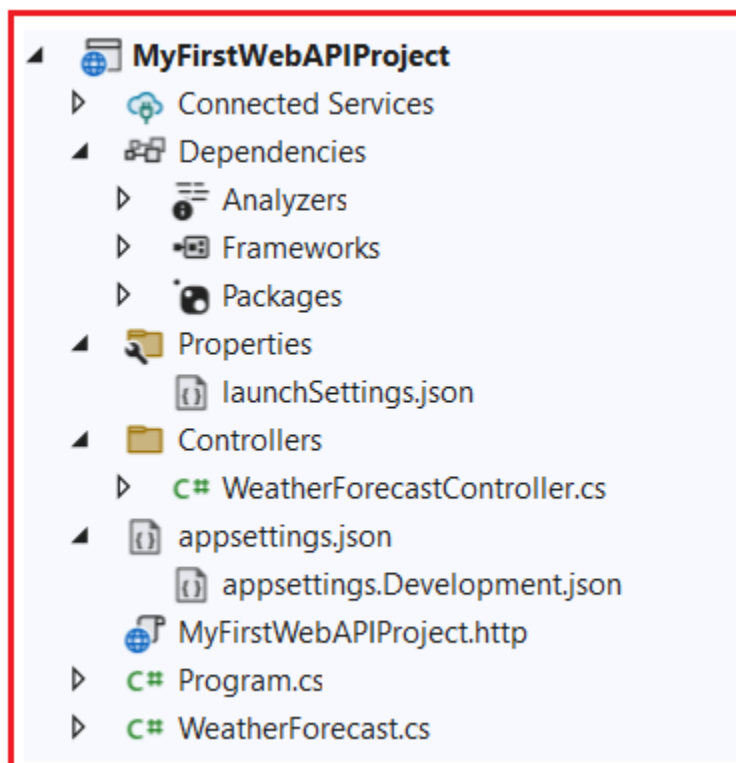
Default ASP.NET Core Web API Files and Folders

In this article, I will discuss the **Default ASP.NET Core Web API Files and Folders**, i.e., those created by default when we create a new ASP.NET Core Web API Application. Please read our previous article discussing [How to Create, Build, and Run ASP.NET Core Web API](#) in Visual Studio.

When you create a new **ASP.NET Core Web API project** in **Visual Studio 2022 with .NET 8**, a predefined set of **files and folders** is automatically generated. This structure serves as the **foundation of the application**, ensuring developers have a well-organized starting point. Understanding the purpose of each folder and file is essential for maintaining clean architecture, integrating external services, and extending the application with new features.

Default ASP.NET Core Web API Project Structure in Visual Studio:

We have created an ASP.NET Core Web API project using Visual Studio 2022, which comes with a predefined set of files and folders as shown in the below image.



Let's proceed and understand what each of these files and folders does and why they're essential.

Connected Services

The **Connected Services** node in Solution Explorer allows developers to easily integrate external services into their applications without manually writing commonly used code for connectivity, authentication, or configuration.

It provides integration for services such as:

- **Azure Services** – Azure Storage, Azure App Services, Azure Key Vault, etc.
- **Third-party REST APIs** – For example, Google APIs, Stripe for payment processing, or Twilio for SMS.
- **Microsoft Office 365 APIs** – Accessing Office 365 resources like emails, calendars, or SharePoint.

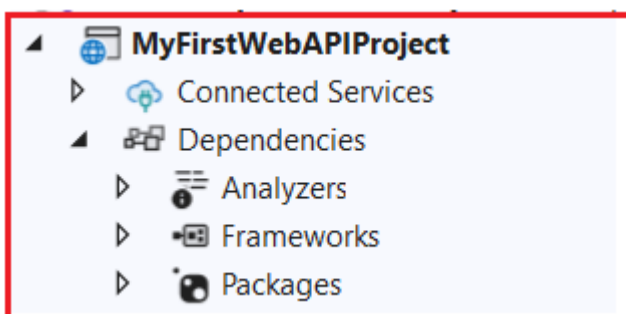
Benefits of Connected Services:

- **Automatic Configuration** – Adds the necessary configuration files and entries to appsettings.json.
- **Client Code Generation** – Creates client libraries that allow developers to call external APIs as strongly typed methods instead of crafting raw HTTP requests.
- **Dependency Management** – Installs required NuGet packages automatically.

Example: If you want to use **Azure Key Vault** for securely storing API keys or database connection strings, you can add it via Connected Services, and Visual Studio will handle package installation, configuration, and secret retrieval setup.

Dependencies:

The **Dependencies** folder shows all **Packages, Analyzers, SDKs, and Frameworks** your project depends on. It ensures that everything needed to build and run your Web API is properly referenced. This makes it easy to see exactly what external components are powering your application. When expanded, it typically contains three key categories: **Analyzers, Frameworks, and Packages**, as shown in the below image.



Let us proceed and understand what each of these folders typically contains:

Analyzers

Analyzers are **Static Code Analysis Tools** that run inside Visual Studio and during build time, it is used to improve code quality. They help with:

- **Code Quality Checks:** Detecting potential bugs, unused variables, or unsafe patterns.
- **Style Enforcement:** Ensuring coding conventions (e.g., naming conventions, spacing rules, formatting, etc.) are followed.
- **Performance Suggestions:** Recommend better coding practices for speed and memory efficiency.

Examples:

If you use **Roslyn analyzers** or install **StyleCop.Analyzers**, they can enforce best practices and provide warnings when rules are violated, keeping your codebase consistent and maintainable.

- Warn if you declare a variable and never use it.
- Enforce camelCase for variables and PascalCase for classes.
- Suggest LINQ optimizations or async improvements.
- An analyzer may suggest using `String.IsNullOrEmpty()` instead of manually checking for null/empty strings.

Frameworks

The Frameworks section lists the core libraries and runtime components that your project depends on. By default, two major framework groups are included. They are as follows:

Microsoft.NETCore.App:

This Includes the .NET runtime, Base Class Libraries (BCL), Garbage Collector, JIT compiler, and other core functionality required to run .NET applications.

- **Base Class Libraries (BCL):**
 - System.IO → File and stream handling.
 - System.Collections → Collections like lists and dictionaries.
 - System.Threading → Multithreading and parallel programming.
- **Core Runtime Libraries:** Provides the **Garbage Collector (GC)**, **JIT Compiler**, and runtime services necessary for executing .NET Core applications.

Microsoft.AspNetCore.App:

Includes all the **ASP.NET Core libraries** required for web development. It provides:

- **Web Server Implementations** → Kestrel for cross-platform hosting and IIS integration for Windows hosting.
- **ASP.NET Core MVC Framework** → For building APIs and web apps using the MVC pattern.
- **Razor Engine** → Used for processing Razor pages and views in dynamic web content.
- **Routing and Model Binding** → Enables mapping of HTTP requests to controllers and automatic data binding.

- **Authentication and Authorization Libraries** → Provides support for securing APIs with JWT, OAuth, Identity, etc.

Without these frameworks, an ASP.NET Core Web API cannot run because they provide the runtime, web server, and request-handling infrastructure.

Packages

The **Packages Section** lists external **NuGet packages** that are added to extend the project's functionality. They are downloaded from NuGet.org and are automatically restored whenever you build the project on a new machine. Packages can be **removed** if no longer required, keeping the project lightweight.

Common examples include:

- **Swashbuckle.AspNetCore** – Integrates **Swagger UI** for interactive API documentation and testing.
- **Entity Framework Core (EF Core)** – Provides ORM (Object Relational Mapping) support for database access.
- **Serilog / NLog** – Advanced logging frameworks for structured and centralized logging.
- **IdentityServer** – A library for implementing authentication, authorization, and token management.

Example: By default, a new Web API project includes **Swashbuckle.AspNetCore** so you can test your endpoints using Swagger UI at runtime without additional setup.

Properties:

The **Properties folder** in an ASP.NET Core project contains files that define how the application behaves during **development and debugging** inside Visual Studio or when running via the .NET CLI. By default, it includes only one file: **launchSettings.json**.

This file is **not used in production**. Instead, it is only relevant during local development, helping Visual Studio and the dotnet run command determine:

- Which URLs the application should listen on.
- Whether it runs on HTTP, HTTPS, or IIS Express.
- Which environment variables should be set (for example, ASPNETCORE_ENVIRONMENT=Development).
- Whether a browser should open automatically and which URL it should open.

launchSettings.json

The **launchSettings.json** file contains launch profiles that specify how the application should be started. Each **profile** is like a separate startup configuration, allowing you to run the same project in different modes (HTTP,

HTTPS, or IIS Express).

A typical structure includes:

- **iisSettings** – Defines IIS Express–specific settings like application URL and SSL port.
- **profiles** – Defines multiple launch profiles (HTTP, HTTPS, IIS Express).

Now, open the launchSettings.json file; by default, you will see the following settings.

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:13420",
      "sslPort": 44318
    }
  },
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5021",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7191;http://localhost:5021",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

```
}  
}  
}  
}
```

Example Default Profiles Include:

1. **HTTP** – Runs the app on plain HTTP.
2. **HTTPS** – Runs the app with SSL enabled for secure communication, usually involving a development SSL certificate.
3. **IIS Express** – Runs the app in IIS Express, simulating a production IIS environment. IIS Express is the lightweight version of IIS used only for development.

HTTP Profile:

The **HTTP profile** configures the application to run over non-secure HTTP (without encryption). This is often the simplest setup for local API testing when security is not a concern. By default, you will get the following settings in the launchsettings.json file for the HTTP Profile:

```
"http": {  
  "commandName": "Project",  
  "dotnetRunMessages": true,  
  "launchBrowser": true,  
  "launchUrl": "swagger",  
  "applicationUrl": "http://localhost:5021",  
  "environmentVariables": {  
    "ASPNETCORE_ENVIRONMENT": "Development"  
  }  
}
```

Default Settings:

- **Application URL** → `http://localhost:<port>` (e.g., `http://localhost:5021`)
- **Launch URL** → `swagger` (automatically opens Swagger UI when the app starts).
- **Environment** → `Development` (sets `ASPNETCORE_ENVIRONMENT=Development`).

Explanation:

- Runs your application on a **localhost port without SSL**.
- Ideal for **quick API testing**, especially when security is not important during development.
- Faster setup, since no certificate or HTTPS handling is involved.

Note: Use this profile for local-only scenarios where you don't need encrypted communication.

HTTPS Profile:

The **HTTPS profile** configures the application to run with **secure communication** using SSL. ASP.NET Core automatically provides a **development-time SSL certificate** when you install the .NET SDK that Visual Studio trusts locally. By default, you will get the following settings in the launchsettings.json file for HTTPS Profile:

```
"https": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "launchUrl": "swagger",
  "applicationUrl": "https://localhost:7191;http://localhost:5021",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
```

Default Settings:

- **Application URLs** → https://localhost:<ssl-port> (e.g., https://localhost:7191) and sometimes also http://localhost:<port> for backward compatibility.
- **Launch URL** → swagger.
- **Environment** → Development.

Explanation:

- Runs the application with **encryption enabled** (HTTPS).
- Suitable for testing **secure endpoints** (e.g., login, payment gateways).
- Helps simulate **real-world production environments**, where HTTPS is mandatory.

Note: Use this profile when you want to ensure that your application works correctly over secure channels.

IIS Express Profile:

IIS Express is a lightweight version of **IIS (Internet Information Services)** bundled with Visual Studio. It allows developers to test applications in an environment similar to production IIS but without requiring full IIS setup. By default, you will get the following settings in the launchsettings.json file for the IIS Express Profile:

```
"iisSettings": {
  "windowsAuthentication": false,
  "anonymousAuthentication": true,
  "iisExpress": {
    "applicationUrl": "http://localhost:13420",
    "sslPort": 44318
  }
}
```

And for IIS Express, it will use the following settings:

```
"IIS Express": {
  "commandName": "IISExpress",
  "launchBrowser": true,
  "launchUrl": "swagger",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
```

Default Settings:

- **Application URL** → Defined under iisSettings (e.g., http://localhost:13420).
- **SSL Port** → 44318 or another randomly assigned port.
- **Launch URL** → swagger.
- **Environment** → Development.

Explanation:

- Launches the app using **IIS Express instead of Kestrel**.
- Allows developers to test features like IIS-specific authentication or deployment-related behaviours.
- It is useful when testing scenarios like IIS hosting, URL rewriting, or Windows authentication.
- Provides a **closer simulation of production IIS environments**, which helps catch issues early.
- Allows developers to validate how their app behaves under IIS Express before deploying to a full IIS server.

Note: Use this profile if your production environment will run on IIS and you want to test compatibility during development.

Authentication in IIS:

The **iisSettings** section in `launchSettings.json` defines how your ASP.NET Core app will run under **IIS Express** during local development. Inside it, we have two important authentication-related flags:

Windows Authentication?

This setting controls whether **Windows Authentication** is enabled in IIS Express. Windows Authentication uses the **Windows credentials** of the logged-in user (domain username/password) to authenticate them. It is typically used in **intranet applications** where users are part of an Active Directory domain.

When set to **true**:

- The application will use the **Windows user account** of the person accessing the application.
- Useful in **enterprise/intranet applications**, where you want to identify users based on their Windows login without requiring them to log in again.

When set to **false**:

- Windows Authentication is disabled.
- The application won't automatically recognize Windows user identities.

Example Use Case: An internal company portal where employees log in to their Windows machines, and the application automatically authenticates them without asking for credentials again.

Anonymous Authentication

This setting determines whether **Anonymous Authentication** is allowed in IIS Express. If enabled, users can access the application **without providing credentials**. The application itself (e.g., via ASP.NET Core Identity or JWT) can still implement its own authentication later.

When set to **true**:

- Anyone can access the application anonymously.
- The app does not attempt to validate user identity via IIS.

When set to **false**:

- Anonymous requests are rejected unless another authentication method (like Windows Authentication) is enabled.

Example Use Case: A public API endpoint that doesn't require authentication, like a health-check endpoint or open API documentation.

Controllers Folder:

The **Controllers folder** is the core of any ASP.NET Core Web API application. It contains the **controller classes**, which are responsible for:

- Accepting **incoming HTTP requests** from clients (e.g., browser, Postman, mobile app).
- Executing the necessary **business logic** (either directly or via service/repository layers).
- Returning the appropriate **HTTP response** back to the client (e.g., JSON data, status codes).

Controllers act as the **bridge between clients (such as browsers, mobile apps, or API consumers)** and our application's business logic.

- Each **controller class** typically represents a **resource** or a **set of related endpoints**. For example, you might have a `ProductsController` to handle product-related APIs or a `UsersController` for user management.
- Controllers use **routing attributes** (like `[Route]`, `[HttpGet]`, `[HttpPost]`) to map specific HTTP requests to action methods.
- Each **action method** inside a controller corresponds to an operation that can be performed on the resource, such as:
 - **GET** → Retrieve data
 - **POST** → Insert new data
 - **PUT** → Update existing data
 - **DELETE** → Remove data

Example: WeatherForecastController

By default, when we create a new ASP.NET Core Web API project, Visual Studio adds a **sample controller** named `WeatherForecastController`. This controller demonstrates the basic structure and functionality of a Web API endpoint.

```
using Microsoft.AspNetCore.Mvc;
namespace MyFirstWebAPIProject.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class WeatherForecastController : ControllerBase
    {
        private static readonly string[] Summaries = new[]
        {
```

```

        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy",
        "Hot", "Sweltering", "Scorching"
    };

    private readonly ILogger<WeatherForecastController> _logger;

    public WeatherForecastController(ILogger<WeatherForecastController>
logger)
    {
        _logger = logger;
    }

    [HttpGet(Name = "GetWeatherForecast")]
    public IEnumerable<WeatherForecast> Get()
    {
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
        {
            Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            TemperatureC = Random.Shared.Next(-20, 55),
            Summary = Summaries[Random.Shared.Next(Summaries.Length)]
        })
        .ToArray();
    }
}

```

appsettings.json file:

The **appsettings.json** file acts as the **primary configuration source** for an ASP.NET Core application. It contains structured key-value pairs in **JSON format** and defines how the application behaves at runtime. Instead of hardcoding settings in code, developers use this file for things like:

- **Database connection strings** (SQL Server, PostgreSQL, etc.).
- **API keys** for third-party services.
- **Logging configuration** (log levels for different parts of the app).
- **Custom configuration values** are defined by the developer.

ASP.NET Core also supports **hierarchical configuration** and **environment-specific overrides**. For example:

- appsettings.json → Base/default settings.
- appsettings.Development.json → Overrides for development environment.
- appsettings.Production.json → Overrides for production deployment.

This layered approach enables developers to have **distinct settings for each environment** while maintaining a consistent codebase.

Default Structure Created by Visual Studio

When you create a new ASP.NET Core Web API project, Visual Studio generates a default `appsettings.json` file with the following content:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Code Explanations:

Let's break this down:

Logging Section

The Logging section controls the **logging behaviour** of the application. Logging is essential for debugging, monitoring, and diagnosing issues.

- **Default: "Information"**
 - Sets the minimum log level for the application.
 - This means *that messages with levels Information, Warning, Error, or Critical will be logged, but those with levels Debug and Trace will be ignored.*
- **Microsoft.AspNetCore: "Warning"**
 - Specifically configures logs coming from Microsoft.AspNetCore namespace.
 - Only messages with level *Warning* or higher (Warning, Error, Critical) will be logged.
 - Reduces noise in logs by filtering out lower-level framework messages.

Example: If a controller logs an **Information** message, it will appear in the logs. However, if ASP.NET Core internals (such as middleware) log an **Information** message, it won't be displayed because it's set to *Warning*.

AllowedHosts

The AllowedHosts setting specifies which hostnames are permitted to send requests to the application.

- “*” → Wildcard that allows **all hosts**. This is convenient during development because you can test APIs from different tools or localhost ports.
- In **production**, this should be restricted to specific domains for **security reasons**.

Example: “AllowedHosts”: “myapi.com;www.myapi.com”

This ensures only requests coming from myapi.com or www.myapi.com are allowed.

Environment-Specific Configuration

ASP.NET Core supports **environment-specific JSON configuration files** that override values in the appsettings.json file.

- **appsettings.Development.json** → Used when running in the Development environment.
- **appsettings.Production.json** → Used when deployed to Production.

Example Scenario:

- In appsettings.json, you define a connection string for development.
- In appsettings.Production.json, you override it with a secure production database connection string.

ASP.NET Core automatically loads the correct configuration based on the **ASPNETCORE_ENVIRONMENT** variable.

Program.cs Class File:

The **Program.cs** file is the **entry point** of every ASP.NET Core application. When the application starts, this is the file that the .NET runtime looks for and executes. It defines the **host configuration**, **service registration**, and the **HTTP request pipeline (also known as the middleware pipeline)**. Together, these determine how your API behaves when handling client requests and responses.

In .NET 6 and later, Microsoft introduced a **minimal hosting model**, which simplified the structure of Program.cs by merging Startup.cs into it. This is why you see all configurations (services + middleware) handled in one place.

Key Responsibilities of the Program.cs

The Program.cs file performs four major tasks:

- **Build and configure the ASP.NET Core host** → Sets up the hosting environment (Kestrel web server, configuration, logging, etc.).

- **Register services** → Adds services such as Controllers, Swagger, Authentication, EF Core, and custom services to the dependency injection (DI) container.
- **Define the middleware pipeline** → Configures request/response processing (e.g., HTTPS redirection, authentication, routing, error handling) and also the order in which the middleware components handle requests.
- **Start the application** → Runs the app (Calls `app.Run()`) so it can begin listening for HTTP requests.

Default Program.cs File

When you create a new Web API project, Visual Studio generates the following default code:

```
namespace MyFirstWebAPIProject
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.

            builder.Services.AddControllers();
            // Learn more about configuring Swagger/OpenAPI at
            https://aka.ms/aspnetcore/swashbuckle
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Service Registration (Dependency Injection)

Service registration is the process of adding reusable components (services) to the **Dependency Injection (DI) container**. ASP.NET Core uses built-in DI by default, which makes it easier to manage dependencies. Services are registered inside the `builder.Services`. These services can then be injected into controllers or other classes.

- **AddControllers()**
 - Registers **MVC controllers** with the DI container.
 - Enables support for Web API endpoints defined in controller classes.
- **AddEndpointsApiExplorer()**
 - Exposes metadata about API endpoints.
 - Required by tools like Swagger/OpenAPI to discover available endpoints.
- **AddSwaggerGen()**
 - Registers services needed to generate **Swagger/OpenAPI documentation**.
 - Provides automatic API documentation and an interactive Swagger UI for testing endpoints.

Example: As your application grows, you can add more services here, such as `AddDbContext()` for Entity Framework Core, `AddAuthentication()` for user security, or `AddCors()` for cross-origin support.

Middleware Configuration

The `app` object defines the **Middleware Pipeline**, a sequence of components that process every incoming HTTP request and outgoing response. Each piece of middleware can inspect, modify, or short-circuit the request/response before passing it to the next component. The order of middleware calls is very important.

- **if (app.Environment.IsDevelopment())**
 - Checks if the current environment is Development.
 - If true, it enables developer-friendly tools like **Swagger** for API testing.
- **app.UseSwagger()**
 - Generates the **Swagger JSON** file that describes your API.
- **app.UseSwaggerUI()**
 - Provides a **web-based Swagger UI** that allows developers to explore and test APIs interactively.
- **app.UseHttpsRedirection()**
 - Redirects all HTTP requests (`http://`) to secure HTTPS (`https://`).
 - Ensures better **security** for API communication.
- **app.UseAuthorization()**
 - Adds **authorization middleware** to enforce security policies.
 - Works in conjunction with authentication middleware to restrict access to endpoints.

- **app.MapControllers()**
 - Maps controller routes to the request pipeline.
 - Without this line, controller endpoints will not be accessible.
- **app.Run()**
 - Starts the application and begins listening for incoming HTTP requests.

Application Startup

Finally:

- **app.Run()** → Starts the web application and begins listening for incoming requests.
- At this point, the app is fully configured, and the middleware pipeline is active.

WeatherForecast.cs class file:

In ASP.NET Core Web API, the **WeatherForecast.cs** file is a simple **model class** that defines the structure of the data returned by the WeatherForecastController. In software design, **models** represent the **shape of the data** your application works with. They are crucial for **data binding** (mapping HTTP request data to C# objects) and for **validation** (ensuring data meets specific rules before processing).

Default Structure of WeatherForecast.cs

When you create a new ASP.NET Core Web API project, Visual Studio generates the WeatherForecast model by default as a sample to demonstrate how controllers return structured data.

```
namespace MyFirstWebAPIProject
{
    public class WeatherForecast
    {
        public DateOnly Date { get; set; }

        public int TemperatureC { get; set; }

        public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);

        public string? Summary { get; set; }
    }
}
```

Key Properties:

- **Date:** The date of the weather forecast.
- **TemperatureC:** Temperature in Celsius.

- **TemperatureF**: Read-only property that converts Celsius to Fahrenheit.
- **Summary**: A brief description of the weather conditions.

Note: For better organization and maintainability, especially in larger projects, consider placing model classes within a dedicated “Models” folder or in a separate class library project.

MyFirstWebAPIProject.http file

The **.http file** in an ASP.NET Core Web API project is a **built-in testing file** that allows you to send HTTP requests directly from within **Visual Studio or Visual Studio Code**. Instead of relying on external tools like **Postman or Fiddler**, or command-line tools such as **cURL**, you can quickly write, send, and debug API requests directly inside your IDE.

This file is part of the **Visual Studio HTTP Client feature** and is automatically generated when you create a new Web API project. It provides a convenient way to test your endpoints during development without needing to switch between tools.

Default Example

When you open the **MyFirstWebAPIProject.http** file for the first time, you will typically see:

```
@MyFirstWebAPIProject_HostAddress = http://localhost:5021

GET {{MyFirstWebAPIProject_HostAddress}}/weatherforecast/
Accept: application/json

###
```

Explanation:

- **@MyFirstWebAPIProject_HostAddress** → A variable holding the base URL of your API (localhost + port).
- **GET {{MyFirstWebAPIProject_HostAddress}}/weatherforecast/** → Defines a sample GET request to the WeatherForecast endpoint.
- **Accept: application/json** → Adds a request header that tells the server to return the response in JSON format.

How to Use

To test an API request using the .http file:

1. **Run the project** → Start your API using the HTTP launch profile in Visual Studio.

2. **Ensure the port matches** → The port number in `MyFirstWebAPIProject.http` must match the one in `launchSettings.json`. If your app runs on `http://localhost:5021`, make sure the variable points to the same URL.
3. **Send request inside IDE** → Open the `.http` file and hover over the request. Visual Studio will show a “**Send Request**” link above it.
4. **View the response** → After clicking “Send Request,” the response (status code, headers, body) will appear in a new results pane or side window.

Once you click on the **Send Request** link, the responses are typically shown in a pane next to the request or in a separate window, allowing you to view the status code, response body, and headers, as shown in the image below.

The **.http file** is a developer-friendly tool built into Visual Studio that allows you to **send and test API requests directly inside your IDE**. It simplifies debugging, supports multiple HTTP methods, and provides quick access to responses without relying on external tools.

Note: As we progress in this course, we will see how to perform GET, POST, PUT, PATCH, and DELETE requests using this .http file to test our APIS.

The default files and folders created in an ASP.NET Core Web API project provide a well-structured foundation for building modern web applications. Each component, whether it is configuration files like appsettings.json, essential classes like Program.cs, or testing utilities like the .http file, has a specific role in simplifying development and ensuring maintainability. By understanding the purpose of these default elements, developers can confidently

extend the project with new controllers, models, services, and middleware while maintaining a clean and organized architecture.

In the next article, I will discuss the **ASP.NET Core Web API Project File**. In this article, I explain the **Default Files and Folders of the ASP.NET Core Web API Project**. I hope you enjoy the article on Default Files and Folders of the ASP.NET Core Web API Project.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career.

Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “Default ASP.NET Core Web API Files and Folders”

NAVEEN BAGHEL

DECEMBER 17, 2021 AT 11:07 PM

I cant believe the level of information the Quality of the explanation. Just brilliant...

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information